DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

CO1 – ASSIGNMENT 2
Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Course Code & Name	**CSA06 – Design and Analysis of Algorithms**
Assessment	**CO1 – Assignment 2: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System**
Course Outcome Assessed	**Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.**
Maximum Marks	**50**
Duration	**1 Week**
Mode	**Individual**
Student Reg. No.	**192411048**
Student Name	**CH.Purnima**

1. Introduction and Problem Understanding

The assignment considers a university student-record database in which every student is identified by a unique Registration Number. The database may grow from about 1,000 records to 10,000, 100,000 and eventually 1,000,000 or more records. The central requirement is to determine whether a registration number exists and, when it exists, retrieve the corresponding record efficiently.

The assignment brief explicitly requires at least three meaningfully different strategies, including at least one iterative and one recursive approach. It also requires the analysis to include preprocessing such as sorting, mathematical derivations, best/average/worst cases, asymptotic notation, a recurrence relation, implementation and experimental validation, followed by a recommendation and a redesign decision for 50,000 searches per hour on one million records. These requirements are followed in the sections below.

Chosen strategies:

- Strategy 1 – Linear Search: sequentially inspect records. No preprocessing is required.
- Strategy 2 – Iterative Binary Search: sort the records by Registration Number and repeatedly halve the search interval.
- Strategy 3 – Recursive Binary Search: the same binary-search decision process expressed recursively. This satisfies the required recursive approach; it is analysed separately for recursion depth and recurrence, while the underlying search strategy is not treated as a fourth fundamentally different algorithm.
- Strategy 4 – Hash Table Lookup: restructure the records into a hash table keyed by Registration Number. This is included as a practically important alternative for the final design decision.

The first three strategies directly satisfy the minimum design requirement. Hashing is added because the updated workload contains very frequent searches and infrequent updates, making preprocessing especially valuable.

2. Q1 – Algorithm Design

2.1 Strategy 1 – Linear Search

Design strategy: brute-force / sequential search. The algorithm starts at the first record and compares its Registration Number with the target. It stops immediately on a match; otherwise it continues until all records have been examined.

Preprocessing: none. The records can remain in their original order.

```
LINEAR-SEARCH(A, key)
1. for i ← 0 to n − 1
2.     if A[i].RegistrationNo = key
3.         return A[i]
4. return NOT-FOUND
```

Why investigate it? It is the simplest baseline and is useful for showing how an O(n) method becomes expensive when the database grows to one million or more records.

2.2 Strategy 2 – Iterative Binary Search

Design strategy: decrease-and-conquer. The records are sorted once by Registration Number. At each step, the middle element is compared with the target. Half of the remaining search interval is discarded.

Preprocessing: sorting. For n records, comparison-based sorting is typically $\Theta(n \log n)$. This cost must be counted whenever the sorted structure has to be built or rebuilt.

```
BINARY-SEARCH-ITERATIVE(A, key)
1. low ← 0, high ← n − 1
2. while low ≤ high
3.     mid ← floor((low + high) / 2)
4.     if A[mid].RegistrationNo = key
5.         return A[mid]
6.     else if A[mid].RegistrationNo < key
7.         low ← mid + 1
8.     else
9.         high ← mid − 1
10. return NOT-FOUND
```

Why investigate it? Binary search gives logarithmic search time after sorting, making it a strong option for large read-heavy datasets when a sorted representation is acceptable.

2.3 Strategy 3 – Recursive Binary Search

Design strategy: recursive decrease-and-conquer. Each call solves one half of the current interval. The recursion terminates when the interval becomes empty or the middle record matches the target.

```
BINARY-SEARCH-RECURSIVE(A, key, low, high)
1. if low > high
2.     return NOT-FOUND
3. mid ← floor((low + high) / 2)
4. if A[mid].RegistrationNo = key
5.     return A[mid]
6. if key < A[mid].RegistrationNo
7.     return BINARY-SEARCH-RECURSIVE(A, key, low, mid − 1)
8. else
9.     return BINARY-SEARCH-RECURSIVE(A, key, mid + 1, high)
```

Preprocessing is the same as for iterative binary search: the input must already be sorted. The main additional consideration is recursion-stack space, which is $\Theta(\log n)$.

2.4 Strategy 4 – Hash Table Lookup

Design strategy: direct-access / hashing. Each record is placed in a hash table using Registration Number as the key. A search computes the key's hash value and checks the corresponding bucket or slot.

Preprocessing: building the hash table requires $\Theta(n)$ expected time and $\Theta(n)$ extra space. Collision handling is necessary. With a good hash function and controlled load factor, expected lookup is $\Theta(1)$, although the worst case can degrade to $\Theta(n)$.

```
BUILD-HASH-TABLE(records)
1. create empty table H
2. for each record r in records
3.     H[hash(r.RegistrationNo)] ← r
4. return H

HASH-SEARCH(H, key)
1. locate the bucket/slot for hash(key)
2. inspect the matching entry/chain
3. return record if key matches; otherwise NOT-FOUND
```

3. Q2 – Basic Operation

Algorithm	Basic operation	Where it occurs	Why it controls time	Growth with n
Linear Search	Registration-number equality comparison	Inside the loop for every examined record	Each comparison decides whether the target has been found	Up to n comparisons; $\Theta(n)$ worst case
Iterative Binary Search	Registration-number comparison with middle element	Each loop iteration	Each comparison removes roughly half of the remaining interval	At most floor(log2 n)+1 iterations
Recursive Binary Search	Registration-number comparison at the middle	One comparison per recursive call	Each call reduces the problem to one half	At most $\Theta(\log n)$ calls
Hash Table	Hash computation + key equality check	Bucket/slot lookup and collision resolution	Determines which record/bucket must be inspected	Expected $\Theta(1)$; worst $\Theta(n)$ under severe collisions

For fair comparison, the dominant key comparison is counted for the three comparison-based searches. Hashing has an additional constant-time expected hash computation, so its expected search cost remains constant with respect to n when the table is appropriately sized.

4. Q3 – Derive the Time-Complexity Function Mathematically

4.1 Linear Search

Let n be the number of records. If the target is absent, every record is examined, so the equality comparison executes n times. Therefore, the search cost can be represented as $T_L(n) = c_1 n + c_2$, where c_1 is the cost of one comparison and c_2 represents loop/return overhead. Thus $T_L(n) = \Theta(n)$. If sorting is not performed, total cost is simply $\Theta(n)$ for each search.

Best case: one comparison. Average case under a uniformly random successful position is $(n+1)/2$ comparisons. Worst case: n comparisons.

If k searches are performed, the worst-case search work is $k \cdot \Theta(n) = \Theta(kn)$. This becomes significant when both n and k are large.

4.2 Iterative Binary Search

After sorting, each comparison approximately halves the remaining interval. Starting with n elements, after j iterations the remaining size is approximately n/2^j. The search ends when n/2^j ≤ 1. Hence 2^j ≥ n and $j \geq \log_2 n$. More precisely, the number of iterations is at most $\text{floor}(\log_2 n)+1$.

Search cost: $T_B(n) = c_1 \lfloor \log_2 n \rfloor + c_2 = \Theta(\log n)$. If sorting is required once, the total preparation-plus-one-search cost is $\Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$. For k searches on the same sorted dataset, total cost is $\Theta(n \log n + k \log n)$.

The important practical distinction is that sorting is paid once and can be amortised over many searches.

4.3 Recursive Binary Search

Each recursive call performs constant work and creates one subproblem of size approximately n/2. Therefore the recurrence is $T_R(n) = T_R(n/2) + c$ for $n>1$, with $T_R(1) = c_0$. Expanding gives $T_R(n) = T_R(n/2^k) + kc$. Setting $n/2^k = 1$ gives $k = \log_2 n$, so $T_R(n) = c_0 + c \log_2 n = \Theta(\log n)$.

The search time has the same asymptotic order as iterative binary search, but recursion introduces stack overhead. The recursion depth is $\Theta(\log n)$, so auxiliary stack space is $\Theta(\log n)$.

4.4 Hash Table Lookup

Building the table processes each of n records once, giving expected preprocessing $T_{H,build}(n) = c_1 n + c_2 = \Theta(n)$. For a well-sized table, the expected number of entries inspected in a bucket is bounded by a small constant, giving expected search $T_{H,search}(n) = \Theta(1)$. Therefore, for k searches, expected total work is $\Theta(n+k)$. The worst case is $\Theta(n)$ for one search if many keys collide.

This makes hashing especially attractive when the same database is searched many times and changes relatively infrequently.

5. Q4 – Best, Average and Worst-Case Behaviour

Algorithm	Best case	Average case	Worst case	Reason
Linear	1 comparison, target at first	≈(n+1)/2 comparisons for	n comparisons	Search may need to scan almost/all

	record	uniformly random successful target		records
Binary – iterative	1 comparison, target is at middle	Θ(log n)	Θ(log n)	Interval is halved each iteration
Binary – recursive	1 comparison, target is at middle	Θ(log n)	Θ(log n)	Same halving behaviour; recursion changes implementation
Hash table	Expected O(1), target in its bucket/slot	Expected O(1)	O(n)	Performance depends on hashing and collision distribution

The difference between cases is driven by where the target occurs and, for hashing, how keys distribute across buckets. For binary search, even an unsuccessful search takes only logarithmically many comparisons because every failed comparison eliminates half of the remaining candidates.

6. Q5 – Asymptotic Analysis

Algorithm	**Preprocessing**	**Search (expected/typical)**	**Worst-case search**	**Space**
Linear	Θ(1)	Θ(n)	Θ(n)	Θ(1) extra
Binary iterative	Θ(n log n) sorting if needed	Θ(log n)	Θ(log n)	Θ(1) extra after sorting
Binary recursive	Θ(n log n) sorting if needed	Θ(log n)	Θ(log n)	Θ(log n) recursion stack
Hash table	Θ(n) expected build	Θ(1) expected	Θ(n)	Θ(n)

At n=1,000,000, $\log_2 n$ is about 19.93. Thus a binary search needs only about 20 interval decisions in the worst case, whereas linear search may perform one million comparisons. Hashing can reduce expected lookup work further to a small constant, at the cost of extra memory and a preprocessing/build phase.

Asymptotic notation therefore describes a major scaling difference: Θ(n) grows directly with the number of records, Θ(log n) grows very slowly, and expected Θ(1) remains approximately stable as n grows.

7. Q6 – Formulate the Recurrence Relation

For recursive binary search, the original problem contains n sorted records. One comparison is performed outside the recursive call. Exactly one recursive call is made, and the selected subproblem contains at most ⌊n/2⌋ records. The base case occurs when low > high (empty interval) or the middle record matches.

$T(n) = T(\lfloor n/2 \rfloor) + c, \quad n > 1$
$T(1) = c_0$
where:
• n = number of candidate records in the current interval
• one recursive call is made
• the subproblem has approximately n/2 records
• c = constant work for index calculation and comparison
• c_0 = constant base-case cost

8. Q7 – Solve the Recurrence Relation
The recurrence can be solved by iteration because each recursive level has a simple constant cost and halves the input.

$T(n) = T(n/2) + c$
$= [T(n/4) + c] + c$
$= T(n/4) + 2c$
$= T(n/8) + 3c$
...
$= T(n/2^k) + kc$

At the base case:
$n/2^k = 1$
$2^k = n$
$k = \log_2 n$

Therefore:
$T(n) = T(1) + c \log_2 n$
$= c_0 + c \log_2 n$
$= \Theta(\log n)$

The Master Theorem gives the same result. Here a=1, b=2 and f(n)=Θ(1). Since n^{log_b a}=n^0=1 and f(n)=Θ(1), the recurrence falls under the first Master Theorem case, giving Θ(log n). The result means that doubling the dataset increases the maximum number of recursive levels by only one.

9. Q8 – Implement and Test the Algorithms
The following Python implementation directly corresponds to the algorithms analysed above. The test data use Registration Numbers represented by consecutive integers. The search key −1 is deliberately absent for worst-case comparison-based searching. For the hash table, the same keys are stored in a dictionary-like hash structure.

```
def linear_search(records, key):
    for record in records:
        if record == key:
```

```
            return True
    return False

def binary_search_iterative(records, key):
    low, high = 0, len(records) - 1
    while low <= high:
        mid = (low + high) // 2
        if records[mid] == key:
            return True
        elif records[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return False

def binary_search_recursive(records, key, low=0, high=None):
    if high is None:
        high = len(records) - 1
    if low > high:
        return False
    mid = (low + high) // 2
    if records[mid] == key:
        return True
    if key < records[mid]:
        return binary_search_recursive(records, key, low, mid - 1)
    return binary_search_recursive(records, key, mid + 1, high)

def hash_search(table, key):
    return key in table
```

Test conditions:

- Datasets: 1,000; 10,000; 100,000; and 1,000,000 records.
- All comparison-based searches use the same sorted integer-key dataset.
- An absent target is used for worst-case search behaviour.
- Each timing is averaged over repeated runs; the one-million-record case uses fewer repetitions to keep the experiment practical.
- Hash-table construction is measured separately because it is preprocessing rather than a per-query operation.

Correctness check: for every dataset, all methods return NOT-FOUND for the absent key and return FOUND when an existing key is supplied. This confirms that the implementations correspond to the intended search strategies.

10. Q9 – Experimental Performance

The following measurements were generated by executing the implementations in this document on synthetic datasets in the current Python runtime. Execution time is hardware- and runtime-dependent; therefore the values should be treated as experimental evidence for the observed growth pattern, not as universal benchmark constants.

n	Linear (ms)	Binary Iter. (ms)	Binary Rec. (ms)	Hash (ms)
1,000	0.017	0.001	0.001	0.000
10,000	0.165	0.001	0.002	0.000
100,000	1.888	0.002	0.003	0.000
1,000,000	20.370	0.005	0.006	0.001

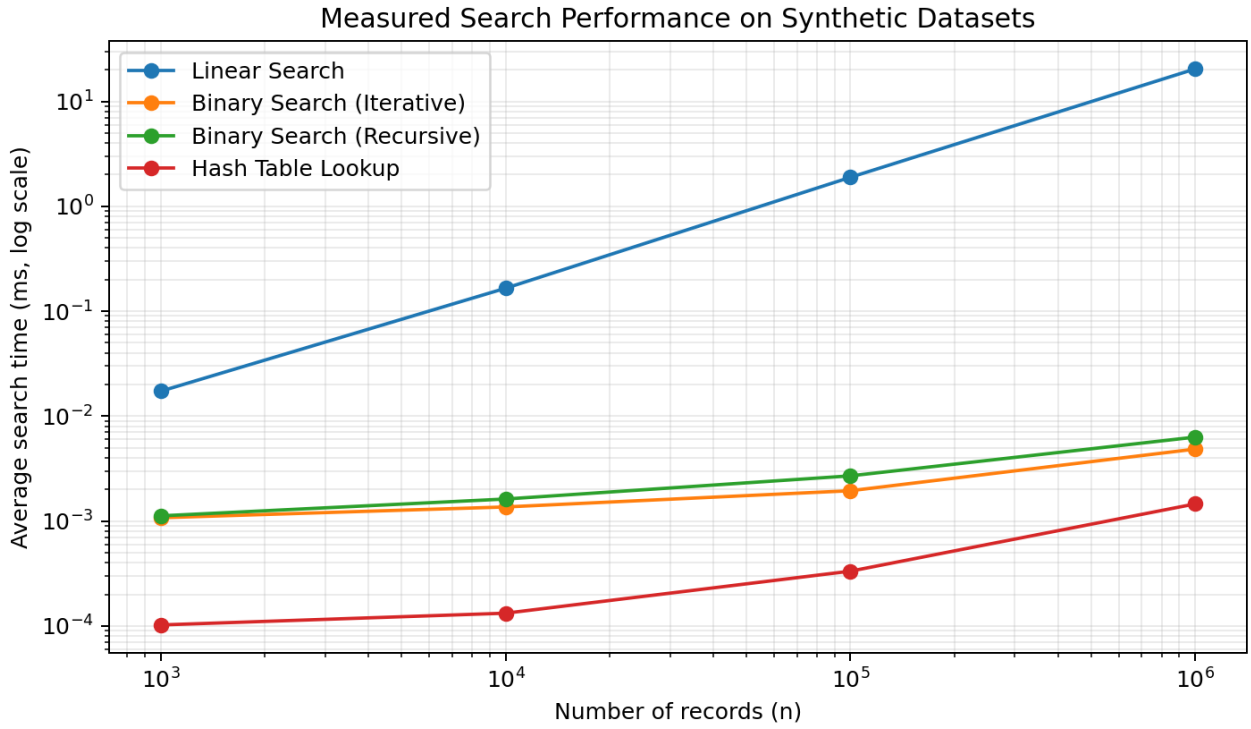


Figure 1. Experimental search-time comparison (logarithmic axes).

The exact timings vary with processor speed, Python version, system load, memory hierarchy and implementation details. The more important observation is the growth pattern: linear search rises substantially with n, while both binary-search implementations increase very slowly. Hash lookup remains approximately constant for the tested absent-key queries.

11. Q10 – Compare Theory with Experimental Behaviour

The experimental behaviour is consistent with the theoretical analysis. Linear search shows approximately proportional growth because an unsuccessful query examines every record. Binary search remains almost flat on the logarithmic plot because its number of comparisons increases only

with $\log_2 n$. The iterative and recursive versions have similar asymptotic behaviour because both repeatedly reduce the candidate set by half.

Small deviations from ideal theoretical curves are expected. Constant factors such as Python loop overhead, function-call overhead in recursion, CPU cache behaviour, memory allocation and operating-system scheduling affect measured time. In particular, recursive binary search can be slightly slower than iterative binary search because each recursive call has additional function-call and stack-management overhead, even though both are $\Theta(\log n)$.

Hash lookup is fast in expected terms because the table provides direct access to a small bucket/slot set. However, the memory cost and preprocessing time are not visible if only the lookup time is measured. A fair system-level decision must therefore consider build time, memory, update frequency and query volume together with lookup time.

12. Q11 – Recommend and Defend the Final Design

Recommended design: use a hash table indexed by Registration Number for the operational search service, with a sorted representation retained when ordered traversal, range queries or audit/reporting requirements make it useful.

The recommendation is based on the workload rather than Big-O alone. The later challenge specifies 50,000 searches per hour on 1,000,000 records, while records are updated only once per day. This is a strongly read-heavy workload. A one-time $\Theta(n)$ hash-table build can be amortised across a large number of searches, producing expected $\Theta(1)$ lookup time.

Criterion	Linear Search	Binary Search	Hash Table
Preprocessing	None	$\Theta(n \log n)$ sorting	$\Theta(n)$ expected build
Expected lookup	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$
Memory overhead	Very low	Low after sorting	Higher
Ordered/range access	Natural only if source is ordered	Excellent	Not naturally ordered
Frequent searches	Poor at large n	Very good	Excellent
Update handling	Simple	May require maintaining order	Expected O(1) insertion/update
Main limitation	Scales poorly	Needs sorted data	Collision/memory considerations

13. Q12 – Day-4 Design Challenge

13.1 (a) Impact Analysis

The workload has changed from a general search problem to a clearly read-heavy operational workload: 50,000 search requests per hour on 1,000,000 records. This is about 13.89 searches per second on average. New records are added only once per day, so the database structure does not need to be rebuilt after every query and frequent updates are not the dominant cost.

The new conditions make preprocessing more attractive because a build cost can be amortised across many searches. They also make an O(n) per-query method unsuitable: in the worst case, 50,000 searches could each inspect up to 1,000,000 records.

13.2 (b) Evaluation of the Original Design

Linear search should be rejected for this workload. Its worst-case hourly search work is approximately 50,000 × 1,000,000 = 50,000,000,000 record comparisons, ignoring other overhead.

Binary search is much better. Once sorted, each query requires at most about $\log_2(1{,}000{,}000) \approx 19.93$, or roughly 20 comparisons. The daily sorting/build cost is therefore small relative to the total number of queries, although sorting a million records is more expensive than building a hash index.

Hashing is particularly suitable because the update frequency is low. Building a one-million-entry index once per day and then serving 50,000 searches per hour gives expected constant-time lookups. The preprocessing cost is amortised across the day's requests.

13.3 (c) Adaptation or Redesign

Decision: MODIFY/REDESIGN the original solution into a hash-indexed search service, while retaining a sorted copy or sorted index if ordered reports are required.

DAILY-INDEX-UPDATE
1. Read new/changed student records.
2. Insert or update each Registration Number in the hash index.
3. Validate uniqueness and index consistency.
4. Publish the refreshed index for search requests.

SEARCH(key)
1. Compute hash(key).
2. Locate the corresponding bucket.
3. Compare the key with the candidate entry/entries.
4. Return the matching student record or NOT-FOUND.

Complexity after redesign: daily index construction is $\Theta(n)$ expected for the full build, or $\Theta(u)$ expected for u newly changed records when incremental updates are used. Each search is expected $\Theta(1)$. For k searches, the expected operational cost is $\Theta(k)$, plus the one-time/incremental indexing cost. Memory usage is $\Theta(n)$.

(d) Evidence-Based Defence

Factor	Evidence	Design implication
Dataset size	1,000,000 records	Avoid $\Theta(n)$ per query
Search volume	50,000/hour ≈ 13.89/second	Lookup cost dominates; constant expected lookup is valuable
Update frequency	New records only once/day	Preprocessing can be amortised
Linear search	Up to 50 billion comparisons/hour in worst case	Not practical
Binary search	≈20 comparisons/query after	Practical and predictable

	sorting	
Hash index	Expected O(1) query, O(n) build	Best exact-key lookup choice
Trade-off	Extra memory + collision management	Acceptable for a read-heavy system

13. Conclusion

This assignment compared linear search, iterative binary search, recursive binary search and hash-table lookup for a university student-record system. Linear search requires $\Theta(n)$ work in the worst case and becomes unsuitable as the database grows. Binary search reduces search time to $\Theta(\log n)$ after sorting, with the recursive form satisfying the recursive-design requirement and producing the recurrence $T(n)=T(n/2)+\Theta(1)$. Hashing provides expected $\Theta(1)$ exact-key lookup after $\Theta(n)$ expected preprocessing, at the cost of additional memory and collision management.

The experimental results support the theoretical growth patterns. For the final workload of 1,000,000 records and 50,000 searches per hour with only daily additions, a hash-indexed design is the strongest choice for exact Registration Number retrieval because its build cost can be amortised over a large number of searches. Binary search remains an important alternative when predictable logarithmic performance, sorted order or range-oriented operations are required.